

FirstKnock Sales OS — Ingestion Engine Optimization: Architectural Decisions & Trade-offs

FirstKnock Sales OS Architecture & Optimization Memo

TO:FirstKnock Engineering Leadership & Product Stakeholders**FROM:**Engineering Architecture Team**DATE:**2026-03-14**RE:**Property Ingestion Engine Optimization — Architectural Decisions & Trade-offs

1. Executive Summary

This memo outlines the critical architectural transformations required to achieve our three primary system goals: returning **100%** data accuracy, securing a **30%** API cost reduction, and enabling frontend visualization for over **150,000** properties without browser degradation^{[8][23][25]}. By migrating to a delta-pull ingestion model, hybrid spatial indexing, and WebGL-accelerated rendering, we eliminate our existing physical bottlenecks^{[8][19][24]}. Execution of this strategy guarantees high-throughput map rendering and stabilizes our data ingestion infrastructure against upstream API caps^{[12][14]}.

2. Key Architectural Decisions

Decision 1: Deduplication — Probabilistic/Fuzzy matching vs. continued address_hash

- **What we chose:**Implementation of USPS CASS normalization and probabilistic fuzzy matching (Jaro-Winkler) with strict MLS vs. County arbitration^{[1][2][3]}.
- **What we rejected:**Continuing to use rigid, exact-match address_hash scripts.
- **Why:**Relying on strict hashes causes fragmented duplicate records due to minor formatting discrepancies; probabilistic matching guarantees the highest degree of accuracy for true legal and market status^{[1][3]}.

Decision 2: API Cost — CDC delta-sync state machine vs. continued full-ZIP refreshes

- **What we chose:**A Change Data Capture (CDC) state machine leveraging timestamp filters (lastUpdated_gte) to pull only altered records^{[8][17]}.
- **What we rejected:**Performing daily full-refreshes of high-volume ZIP codes.
- **Why:**Migrating to delta-pulls cuts our RentCast API calls by **85%**, dropping monthly processing costs from **\$450** to **\$67.50** per **100** zip codes tracked^{[7][8]}.

Decision 3: Boundary Accuracy — H3 k-ring pre-filter + PostGIS exact join vs. pure H3 or pure PostGIS

- **What we chose:**A hybrid query model that applies an H3 Level **9** coarse filter before executing an exact PostGIS ST_Covers spatial join^{[6][19]}.

- **What we rejected:**Running pure exact PostGIS geometric operators, or relying solely on coarse H3 spatial approximations ^{[5][12]}.
- **Why:**Executing precise native PostGIS functions on **100,000+**bounds unconditionally triggers massive CPU spikes and server lock-ups; combining both accelerates joins by **400x** ^{[6][12]}.

Decision 4: Caching — Property-level Redis with type-specific TTLs vs. zip-level or no caching

- **What we chose:**A structural Cache-Aside pattern via Redis using strict TTLs (**1-4 hours**for active, **7 days**for off-market) paired with Bloom filters ^{[10][11][18]}.
- **What we rejected:**Broad ZIP-level cache buckets or hitting external APIs per direct customer query.
- **Why:**Granular property TTLs and Bloom filters serve as shields, preventing redundant paid external fetches for overlapping boundaries and securing sub-millisecond data delivery to the frontend UI ^{[9][13]}.

Decision 5: Frontend — MapLibre GL JS + martin tile server vs. Mapbox (licensing cost) or continued Leaflet

- **What we chose:**Replacing standard JSON payloads with the Rust-based Martin dynamic vector tile server, rendered client-side via WebGL-accelerated MapLibre GL JS and Deck.gl ^{[26][28][29][30]}.
- **What we rejected:**Paying for Mapbox's expensive proprietary commercial license, or retaining our existing Leaflet DOM-based rendering ^{[23][32]}.
- **Why:**Leaflet systematically crashes the browser at **100,000**DOM elements, while MapLibre/Deck.gl scales past **1,000,000**data points rapidly without the proprietary overhead costs Mapbox enforces ^{[23][25][32]}.

3. Trade-offs & Risks

- **Complexity increase:**Upgrading deduplication to probabilistic matching requires integrating data normalization (USPS CASS) and ongoing engineering calibration of fuzzy match thresholds to avoid false positive merges ^{[1][2]}.
- **CDC state machine adds operational complexity:**Managing timestamp high-watermarks creates monitoring challenges; if properties are purely deleted upstream (rather than updated), they will be missed by the delta-pull, requiring secondary infrastructure jobs to reconcile stale silent deletions ^{[20][21]}.
- **MapLibre migration requires 8-week parallel run:**Safely porting map functionality requires maintaining the Leaflet `@maplibre/maplibre-gl-leaflet` fallback using feature flags during an 8-week migration, consuming substantial engineering bandwidth ^{[33][34]}.
- **Redis cache adds infrastructure dependency:**Using distributed Deno KV and Redis state introduces a critical infrastructure failure point; if the distributed logic fails, divergent workers will overload the RentCast API and immediately trigger cascading **429 Too Many Requests** blocks ^{[14][15][22]}.

Metric	Baseline	Target	Projected Outcome
Max properties rendered	50,000 (Hard Cap) ^[23]	150,000+	Stable MapLibre WebGL integration (1,000,000+limit) ^[25]
RentCast API Cost	\$450 per 100 zip codes ^[8]	\$67.50 per 100 zip codes ^[8]	85% structural cost reduction ^{[7][8]}
Viewport query latency	>1200ms (Unindexed) ^[31]	<100ms	Hardware efficiency via PostGIS GiST + && operator ^{[27][31]}
Map DOM Elements	~50,000 (UI Lags) ^[25]	0 (Canvas-rendered)	Handled purely via graphics engine (Symbol/Deck.gl) ^[33]

5. Recommended Phasing

Phase 1: Cost Control & Pipeline Stability (Weeks 1-4) The priority is establishing immediate cost control by halting API quota bleeding ^[8]. Implementing the CDC Delta-Pull architecture utilizing the `lastUpdated_gte` dynamic lookback parameter yields the fastest ROI. During this window, we will wrap all RentCast API connections inside a `Deno.serve` distributed Token Bucket with Full Jitter exponential backoff, preventing request cascades and permanently securing our targeted **85%** cost savings natively ^{[14][16][17]}.

Phase 2: Caching & Basic Accuracy (Weeks 5-8) With budget outlays controlled, focus shifts to deploying the secondary Shared Cache layer using a proactive Cache-Aside schema natively guarded by Bloom filters ^{[10][11]}. Simultaneously, the new deduplication engine components will execute against ingest logic, injecting crucial USPS CASS standardization and standardizing data hygiene across the merged MLS and County datasets prior to downstream storage ^{[1][3]}.

Phase 3: Spatial Scale & Advanced Optimization (Weeks 9-16) Closing out the architectural updates requires scaling the visual elements for **150,000+** maps ^{[23][24]}. We will completely remove Map rendering reliance on Leaflet arrays, adopting MapLibre combined natively with Martin tile vectors ^{[28][30]}. The backend will transition full-time to the H3 + PostGIS hybrid boundary mapping standard, supported smoothly by pre-calculated dynamic **P95** recording lag matrices to ensure we systematically collect slow-filing rural county files efficiently without manual intervention ^{[4][6]}.

6. Decision Required

We are requesting immediate stakeholder approval to proceed with the **Phase 1 (Weeks 1-4)** CDC implementation scope, formal budget authorization to provision the necessary Redis shared

caching infrastructure, and explicit leadership sign-off on the **8-week** MapLibre engineering migration timeline [8][14][33]. Your approval empowers the architecture team to lock in vital structural API savings next month while sequentially eliminating our crippling frontend property map bottlenecks [8][25].

References

- [0] "@maplibre/maplibre-gl-leaflet".
<https://www.npmjs.com/package/@maplibre/maplibre-gl-leaflet>
- [0] "How to render 400K points in the browser: a comparison of libraries".
https://www.linkedin.com/posts/mbforr_rendering-spatial-data-in-the-browser-continues-activity-7386028312575242240-x0Pc
- [0] "MapLibre GL JS: The Open-Source Powerhouse for Interactive Maps in Modern Web Apps - BrightCoding".
<https://www.blog.brightcoding.dev/2025/10/05/maplibre-gl-js-the-open-source-powerhouse-for-interactive-maps-in-modern-web-apps>
- [0] "RentCast API Pricing | DiscoverMyBusiness".
<https://discovermybusiness.co/rentcast-api-pricing/>
- [0] "deck.gl vs leaflet vs mapbox-gl vs react-google-maps vs react-map-gl | npm trends".
<https://npmtrends.com/deck.gl-vs-leaflet-vs-mapbox-gl-vs-react-google-maps-vs-react-map-gl>
- [0] "The State of the MapLibre Ecosystem: A 2025 Plugin Analysis - Blog | Geoman.io".
<https://geoman.io/blog/the-state-of-the-maplibre-plugin-ecosystem>
- [0] "Cache-First Geocoding with Azure Maps: Key Topology, TTL Heuristics, and Quota Smoothing - DEV Community".
https://dev.to/daniel_romitelli_44e77dc6/cache-first-geocoding-with-azure-maps-key-topology-ttl-heuristics-and-quota-smoothing-5708
- [0] "Mapbox vs. MapTiler vs. MapLibre vs. Leaflet: Which to Choose? - GIS People".
<https://www.gispeople.com.au/mapbox-vs-maptiler-vs-maplibre-vs-leaflet-which-to-choose/>
- [0] "Why I migrate from Leaflet to MapLibre".
<https://techilearned.com/why-i-migrate-from-leaflet-to-maplibre/>
- [0] "Technical Comparison Analysis of Leaflet and Mapbox in Vector Tile Rendering".
<https://www.oreateai.com/blog/technical-comparison-analysis-of-leaflet-and-mapbox-in-vector-tile-rendering/6cba-def28ad2c5f83cb9c5687ad69ec2>
- [0] "Address Matching Explained: Resident, Household & Individual Matching".
<https://matchdatapro.com/address-matching-explained-resident-household-individual-matching/>
- [0] "chore: release · Pull Request #2278 · maplibre/martin".
<https://github.com/maplibre/martin/pull/2278>
- [0] "Implement fast, space-efficient lookups using Bloom filters in Amazon ElastiCache | Amazon Web Services".
<https://aws.amazon.com/blogs/database/implement-fast-space-efficient-lookups-using-bloom-filters-in-amazon-elasticsearch>

- [0] "LLM Response Caching: Redis vs Cloudflare KV Performance | PropTechUSA.ai".
<https://www.proptechusa.ai/news/llm-response-caching-redis-vs-cloudflare-kv-performance>
- [0] "Tech Matchups: Cache Invalidation Strategies: TTL vs Manual vs Event-driven".
<https://www.swiftorial.com/matchuups/caching/cache-invalidation-ttl-vs-manual-vs-event/>
- [0] "Real Estate Listings: Handling Geo-Spatial Queries Efficiently with PostGIS".
<https://codingclave.com/blog/real-estate-listings-handling-geo-spatial-queries-efficiently-with-postgis>
- [0] "Deduplication reference".
<https://documentation.cluedin.net/management/deduplication/deduplication-reference>
- [0] "Cache Invalidation Strategies: Keeping Your Data Fresh in Redis - DoHost".
<https://dohost.us/index.php/2026/03/12/cache-invalidation-strategies-keeping-your-data-fresh-in-redis/>
- [0] "How we made geo joins 400× faster with H3 indexes".
<https://floedb.ai/blog/how-we-made-geo-joins-400-faster-with-h3-indexes>
- [0] "feat: basic etag handling for tile serving · Pull Request #1834 · maplibre/martin".
<https://github.com/maplibre/martin/pull/1834>
- [0] "Fixing MLS Data Chaos: How to Design RESO-Compliant Real Estate Integrations".
<https://gtcsys.com/fixing-mls-data-chaos-how-to-design-reso-compliant-real-estate-integrations/>
- [0] "Release Notes - RentCast API".
<https://developers.rentcast.io/changelog>
- [0] "API Rate Limiting: How to Handle 429 Errors in Production | API Status Check Blog".
<https://apistatuscheck.com/blog/api-rate-limiting-429-errors-production>
- [0] "The Importance of Data Latency in Real Estate Investing".
<https://batchleads.io/blog/why-is-data-latency-important-in-real-estate-investing>
- [0] "Implementing a Distributed Rate Limiter Using Redis and the Token Bucket Algorithm | by Erwin Hermanto | Jan, 2026 | Medium".
<https://medium.com/%40erwindev/implementing-a-distributed-rate-limiter-using-redis-and-the-token-bucket-algorithm-d3d7748cf34f>
- [0] "spatial relationships - Dr. Balvinder Taneja".
<https://drbtaneja.com/spatial-relationships/>
- [0] "Henry's Dev Journey".
<http://henrychan.tech/incremental-loading-101-timestamp-watermarking-hash-comparisons-and-cdc/>
- [0] "Building a CDC Pipeline with Debezium: 3 Bugs I Hit and 3 Architecture Lessons I Learned | by Adam Chang | Mar, 2026 | Medium".
<https://medium.com/@adams-chang/building-a-cdc-pipeline-with-debezium-3-bugs-i-hit-and-3-architecture-lessons-i-learned-8f17dd699b13>
- [0] "Retry with Exponential Backoff and Jitter | AI Knowledge Library".
<https://knowledgelib.io/software/patterns/retry-exponential-backoff/2026>
- [0] "How to Build Timestamp-Based CDC".
<https://oneuptime.com/blog/post/2026-01-30-data-pipeline-timestamp-based-cdc/view>

- [0] "Rate Limiting and Throttling Strategies for Production APIs: Token Buckets, Sliding Windows, and Distributed Guardrails - AverageDevs".
<https://www.averagedevs.com/blog/rate-limiting-throttling-production-apis>

- [0] "Why Uber Gridified the World: Solving the "Point-in-Polygon" Problem with H3 | by The Speed Engineer | Beyond Localhost | Mar, 2026 | Medium".
<https://medium.com/beyond-localhost/why-uber-gridified-the-world-solving-the-point-in-polygon-problem-with-h3-80c821676f43>

- [0] "Polygon boundaries in PostGIS spatial predicates".
<https://gis.stackexchange.com/questions/460560/polygon-boundaries-in-postgis-spatial-predicates>

- [0] "PostgreSQL Performance Tuning in 2026: Indexes, Query Optimization, and Scaling to Millions of Rows | ZeonEdge".
<https://zeonedge.com/en/blog/postgresql-performance-tuning-2026-indexes-query-optimization>